

RegexOne Class

Dev Gandhi

2026-06-27

Practical Application in I-O Psychology

In the context of advanced HR analytics and I-O psychology, regular expressions (Regex) are an invaluable tool for unstructured text processing. Whether I am parsing through thousands of open-ended employee engagement survey responses, extracting specific behavioral themes for a white paper on quiet quitting, or standardizing messy job titles from disparate HR databases, Regex allows me to automate text classification seamlessly. Rather than relying on manual review, deploying these patterns within R allows me to clean documents, search text, and isolate exactly the strings I need from massive datasets in seconds.

RegexOne Lesson Reproductions & Notes

Lesson 1: An Introduction, and the ABCs

- **Concept:** Matching literal, exact characters.
- **Solution:** `abc`
- **My Notes:** The most basic regex is just typing the exact letters you want to find. It is case-sensitive, so `a` does not equal `A`.

Lesson 1.5: The 123s

- **Concept:** Matching numeric digits.
- **Solution:** `123` or `\d+`
- **My Notes:** The `\d` metacharacter is heavily used in HR data to pull out employee IDs, salaries, or dates. The `+` means “one or more.”

Lesson 2: The Dot

- **Concept:** The wildcard character.
- **Solution:** `...\.`
- **My Notes:** A single dot `.` matches absolutely *any* single character (except a line break). If I actually want to find a literal period in a document, I have to escape it using a backslash like this: `\.`

Lesson 3: Matching Specific Characters

- **Concept:** Character sets (brackets).
- **Solution:** `[cmf]an`
- **My Notes:** Placing characters in brackets `[]` tells the engine to match *any one* of the characters inside. This is great for handling common misspellings in survey responses.

Lesson 4: Excluding Specific Characters

- **Concept:** Negated character sets.
- **Solution:** `[^b]og`

- **My Notes:** Adding a caret `^` inside the brackets means “match anything EXCEPT what is in these brackets.”

Lesson 5: Character Ranges

- **Concept:** Sequential ranges.
- **Solution:** `[A-Z]\w*`
- **My Notes:** Instead of typing every letter, `[A-Z]` matches any uppercase letter, and `[a-z]` matches lowercase. The `\w` shortcut matches any alphanumeric word character.

Lesson 6: Catching Some Zzz’s

- **Concept:** Specific repetitions.
- **Solution:** `waz{3,5}up`
- **My Notes:** Curly braces `{}` let me specify exactly how many times a character should repeat. `{3,5}` means between 3 and 5 times.

Lesson 7: Mr. Kleene, Mr. Kleene

- **Concept:** The Asterisk `*` and Plus `+` modifiers.
- **Solution:** `a+b*c+`
- **My Notes:** The `*` means zero or more repetitions, while the `+` means one or more. This is incredibly useful for parsing variable-length data, like trailing spaces in messy CSV files.

Lesson 8: Characters Optional

- **Concept:** The question mark `?`
- **Solution:** `\d files? found\?`
- **My Notes:** The `?` makes the preceding character optional (meaning it appears 0 or 1 time). Here, it allows the pattern to match both “file” and “files” perfectly.

Lesson 9: All This Whitespace

- **Concept:** Handling spaces, tabs, and line breaks.
- **Solution:** `\d\.\s+abc`
- **My Notes:** `\s` captures any whitespace character. Using `\s+` is the ultimate weapon for document cleaning when HR data imports with weird, irregular spacing between columns.

Lesson 10: Starting and Ending

- **Concept:** Anchors.
- **Solution:** `^Mission: successful$`
- **My Notes:** The caret `^` anchors the match to the absolute beginning of a line, and the dollar sign `$` anchors it to the end. This ensures I match the whole string, not just a chunk in the middle of a sentence.

Lesson 11: Match Groups

- **Concept:** Extracting specific data (Parentheses).
- **Solution:** `^(file_.+)\.pdf$`
- **My Notes:** Putting parentheses `()` around a pattern captures that exact piece of data so it can be extracted later.

Lesson 12: Nested Groups

- **Concept:** Groups inside of groups.

- **Solution:** `^([A-Z][a-z]{2}\s(\d{4}))`
- **My Notes:** You can capture multiple layers of data at once. For example, capturing an entire date “Jan 2026” in one group, while simultaneously capturing just the year “2026” in an inner nested group.

Lesson 13: More Group Work

- **Concept:** Capturing multiple distinct groups.
- **Solution:** `^(\d+)x(\d+)$`
- **My Notes:** Useful for pulling distinct variables out of a single combined string (like extracting “Height” and “Width” from a dimension string like 1920x1080).

Lesson 14: It’s All Conditional

- **Concept:** The OR operator (Pipe).
- **Solution:** `I love (cats|dogs)`
- **My Notes:** The pipe `|` acts as an “OR” statement. This is perfect for text classification when assigning binary tags (e.g., searching for “quit | left | resigned”).

Lesson 15: Other Special Characters

- **Concept:** Tying it all together.
- **Solution:** `.*`
- **My Notes:** Regular expressions are basically a mini-language. Combining escape characters, wildcards, and groups allows for almost infinite ways to structure and clean data before it ever hits a predictive model.